# Immix: A Mark-Region Garbage Collector with Space Efficiency, Fast Collection, and Mutator Performance

Stephen M. Blackburn, Kathryn S. McKinley

---

Szabó Márk

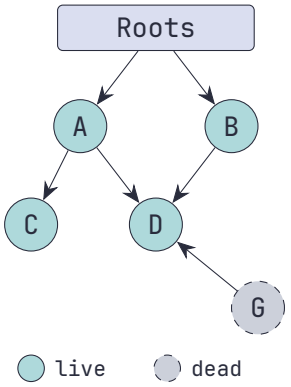
# Garbage Collection Background

# Tracing Garbage Collection

A tracing collector has three jobs:

- **Allocation** of new objects.
- **Identification**: mark every object reachable from the roots.
- **Reclamation** of unmarked objects.

An object is `dead` if we can't reach it from roots.



# The Canonical Tracing Collectors

The paper compares immix against the three canonical tracing collectors:

- **Mark-sweep (MS):** non-moving, objects stay where they were allocated.
- **Semi-space (SS):** moving, copies live objects into a second space.
- **Mark-compact (MC):** moving, slides live objects together within the same space.

## Building blocks

More sophisticated collectors, such as generational ones, are built by combining these.

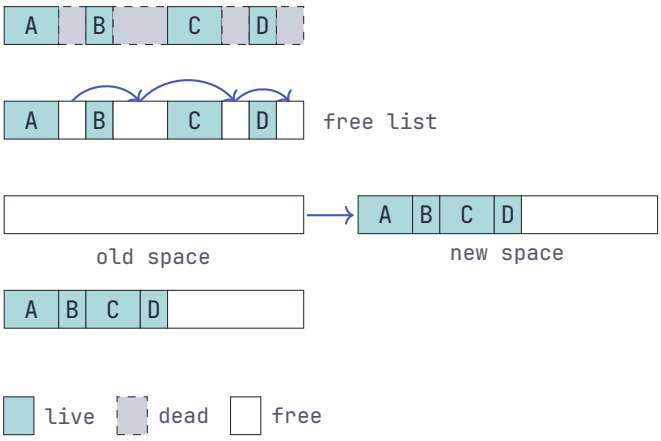
# Three Reclamation Strategies

**After marking**
live and dead objects

**Mark-sweep**
objects stay in place

**Semi-space**
evacuate to a new space

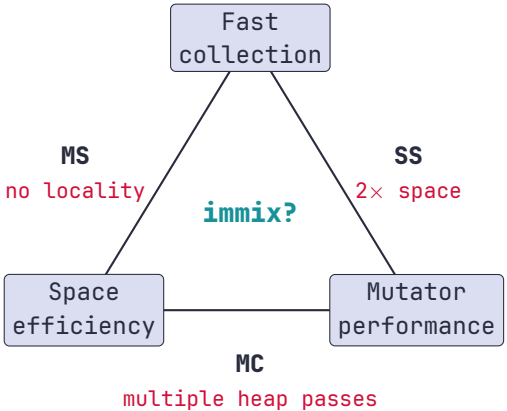
**Mark-compact**
slide to one end



# Three Performance Objectives

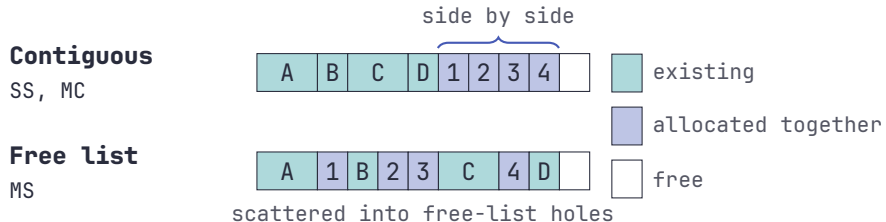
- **Space efficiency:** needs only a small heap.
- **Fast collection:** little time spent collecting.
- **Mutator performance:** the application (*mutator*) runs fast, thanks to locality.

Each canonical collector achieves only two of the three.



# Locality and Contiguous Allocation

- **Contiguous allocation** puts objects allocated at the same time next to each other in space: good **mutator locality**.
- **Mark-sweep** is non-moving and allocates from a free list: no locality for objects allocated at the same time.

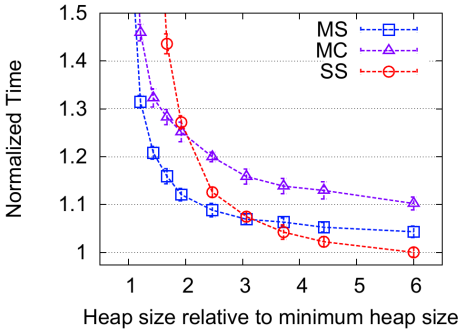


# The Space-Time Trade-off

MS, SS and MC from 1× to 6× the minimum heap:

- **Small heap:** GC runs often, fast collection wins (**MS**).
- **Large heap:** GC runs rarely, fast program wins (**SS**).
- **MC:** collecting is too slow.

The crossing lines are the classic **space-time trade-off**.




(a) Total Time

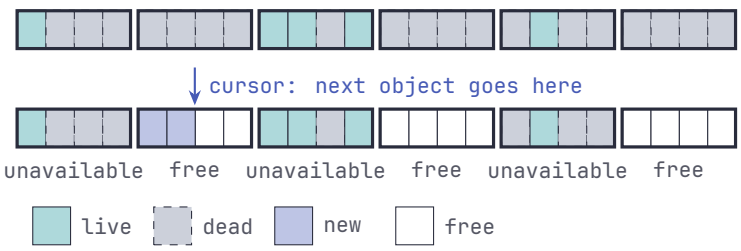
# Mark-Region Collection

# Mark-Region Collectors

- Memory is divided into **fixed-size regions**. Objects can't span regions.
- **Bump allocation:** each new object goes right after the previous one, by advancing (*bumping*) a pointer. When no free region is left, we mark and sweep again.
- A region holding any live object is *unavailable*, all others are *free*.

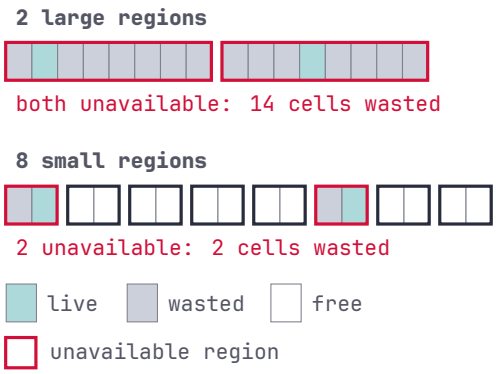
**After marking**
live and dead objects

**After sweep**
whole regions reclaimed



# Two Open Questions

1. **How big should regions be?**
   - **Large:** one small object wastes a whole region.
   - **Small:** less waste, but slower collection.
2. **How to defragment?**
   - A region with **any** live object stays unavailable.

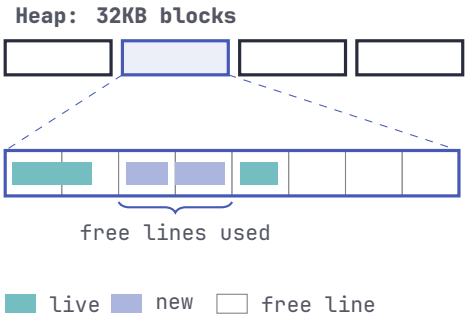


# Immix

# Blocks and Lines

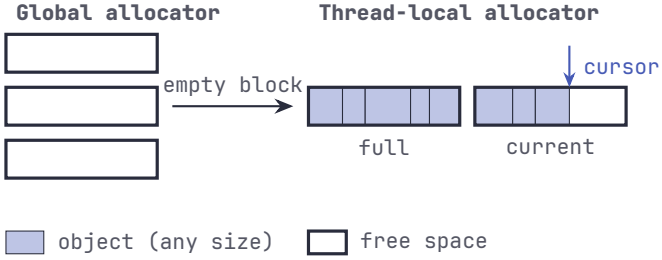
**Answers question 1:** use big blocks when possible, small lines when necessary.

- 32KB **blocks**, split into 128B **lines**.
- Free block: reclaimed **whole**.
- Partly used block: new objects fill its **free lines**.
- Objects span lines, never blocks.



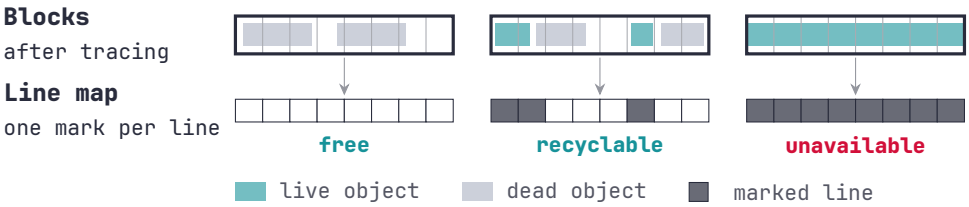
# Initial Allocation

- All blocks start **empty**.
- Each thread's allocator takes an empty block from the **global allocator** and bump allocates into it.
- Block full: take another. No empty blocks left: the heap is full, so a collection starts.



# Identification and Reclamation

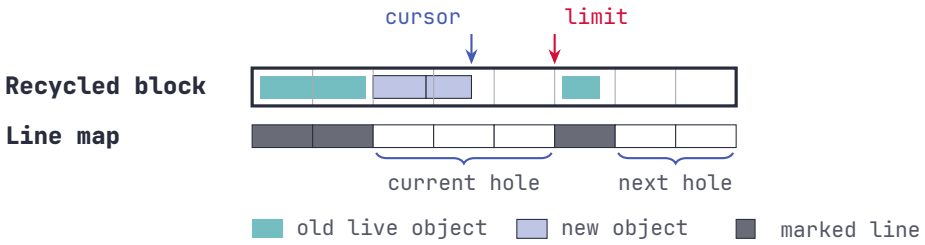
1. **Identification:** trace from the roots and mark each live object **and its lines** in a **line map**.




2. **Reclamation:** read the **line map** and find the entirely free and partially free blocks. Free blocks return to the **global pool** and partially free blocks are **recycled** for the next allocation.

# Steady State Allocation

- Allocate into **recycled blocks** first, in address order. Empty blocks come last.
- Scan the line map for a **hole** (one or more free lines in a row) and bump allocate into it.
- **Limit:** the next marked line or the block's end. Past it: find the next hole, or take a new block.



# Recycling and Allocation Policies

- **Recycling:** a block with at least `F` free lines is recyclable. Simplest `F = 1` worked best.
- **Allocation:** recyclable blocks first, in address order **free blocks** last.



# Optimizations

# Exact Line Marking

- Every line a live object lies on must be marked, or its space could be reused.
- **Exact marking**, for each live object:
  1. read its type to find its **size**,
  2. loop over its memory, marking **each line**.
- **Mark-sweep** just sets **one bit** in the header compared to line marking which is expensive.

**Key observation**

Most objects are smaller than a line (128B), so they span **at most two lines**.

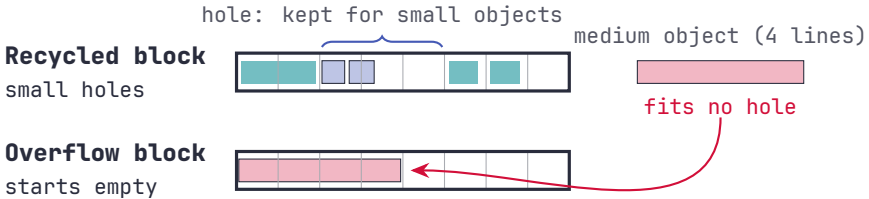
# Conservative Line Marking

- **Small object** (up to a line): mark only its **start line**. No size lookup, one write.
- Its possible second line: the allocator **skips the first line** of every hole.
- **Medium object** (larger than a line): marked exactly, but rare.
- One **header bit**, set at allocation, says small or medium.



# Demand-Driven Overflow Allocation

- **Problem:** a medium object (larger than a line) may fit no hole of a recycled block, so the allocator skips and wastes many holes.
- **Fix:** each allocator also has an **overflow allocator** that only uses empty blocks.



# Opportunistic Defragmentation

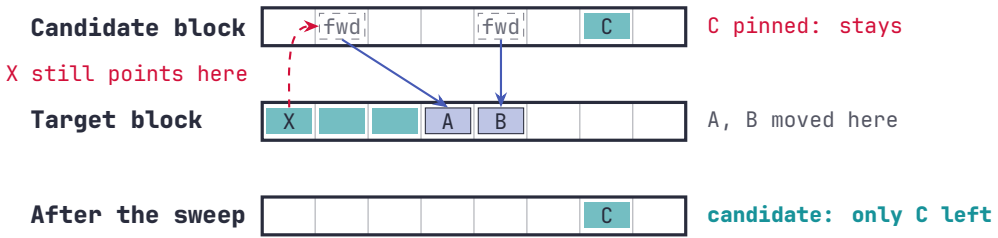
# Opportunistic Evacuation

**Answers question 2:** move objects out of fragmented blocks while marking, only when needed.

- **When:** at the start of a collection, if last time some recyclable blocks went **unused** or too little space was freed.
- **What:** pick the most fragmented blocks as **candidates**, while marking, move their live objects into free space.
- **Opportunistic:** out of space, or the object is pinned? Just mark it in place (plain mark-region behaviour).

# Forwarding Pointers and Pinning

- **Forwarding pointer:** points from the old place to the new one. While tracing, references to the old address are fixed. Afterwards the old place is **freed**.
- **Pinned object:** the program asked for it never to move (e.g. an I/O buffer). A header bit marks it.



# Candidate Selection

- **More holes = more fragmented**: picked first.
- Walk from the most holes down: their **marked lines** add to the space **needed**, their **free lines** stop counting as **available**. Stop when needed > available.

Each row adds up **all blocks** with that many holes:

| Group | Marked | Free | Needed | Available | |
|---|---|---|---|---|---|
| start | | | 0 | 100 | |
| 6 holes | 10 | 15 | 10 | 85 | candidate |
| 5 holes | 20 | 20 | 30 | 65 | candidate |
| 4 holes | 30 | 25 | 60 | 40 | stop |

→ **all** blocks with 6 or 5 holes become candidates.

# Evaluation

# Methodology

*TODO: 20 DaCapo, SPECjvm98 and pjbb2000 benchmarks; Jikes RVM with MMTk; Core 2 Duo, AMD Athlon, PowerPC 970; replay compilation; heaps from 1× to 6× the minimum.*

# Full Heap Performance

*TODO: Immix beats all canonical collectors at every heap size on all three architectures, matching SS mutator time and MS collection time.*

*Copy from the paper: Figure 3 (full heap performance comparison)*

# Full Heap Results at 2× Minimum Heap

*TODO: Per-benchmark slice: immix geomean 0.93 vs MS, worst case 1.01, best 0.82 (jython).*

*Copy from the paper: Table 1 (full heap performance at 2× minimum heap)*

# Space Efficiency

*TODO: Minimum heap size relative to mark-sweep: immix 86%, mark-compact 83%, semi-space far larger.*

*Copy from the paper: Figure 5 (minimum heap sizes)*

# Immix in a Generational Collector

*TODO: Semi-space nursery with an immix mature space (G|SS-IX) vs the production collector G|SS-MS; slightly better on average, 5% better on pjbb2000.*

*TikZ diagram: generational heap: bump-allocated nursery, survivors promoted into the immix mature space*

# Generational Performance at 1.5× Minimum Heap

*TODO: Per-benchmark slice normalized to G|SS-MS: G|SS-IX geomean 0.99, in-place G|IX-IX 1.02, G|MS-MS 1.10.*

*Copy from the paper: Table 2 (generational performance at 1.5× minimum heap)*

# In-Place Generational: Sticky Mark Bits

*TODO: Mark bits stay set between collections, so a nursery collection only traces newly allocated objects without moving anything. A write barrier remembers modified old objects. Supports pinning, unlike a copying nursery.*

*TikZ diagram: heap after a full collection with old objects marked; new unmarked objects allocated between them; a nursery collection traces only the unmarked ones*

# Generational Results

*TODO: Copying generational, in-place (sticky mark bits) generational, and all immix variants compared.*

*Copy from the paper: Figure 6 (generational collector performance)*

# Selected Benchmarks

*TODO: javac and luindex: full heap immix is the best collector. pjbb2000: immix variants beat the production collector.*

*Copy from the paper: Figure 4 (selected benchmarks)*

# What Makes Immix Work

*TODO: Ablation: block-only marking inflates marked space by 93%, lines without defragmentation by 23%. Defragmentation rarely triggers but frees 95% of candidate space. Overflow allocation handles about 4% of allocation.*

# Conclusion

# Related Work

**Mark-region collectors existed**, in JRockit and IBM's JVM, but were never described or evaluated in a publication.

How immix differs:

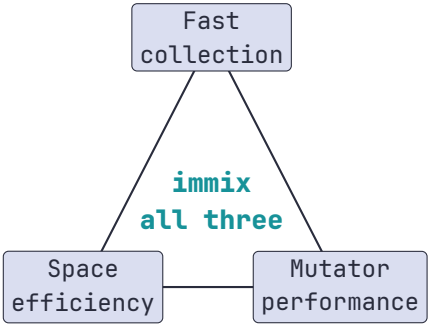
- **Defragmentation:** JRockit and IBM use compaction, immix evacuates opportunistically in a **single pass**.
- **Granularity:** IBM never reuses gaps smaller than 512 bytes, immix reclaims **128-byte lines**.
- **Metadata:** 0.8% of the heap for immix vs 3.1% for IBM.

# Why Immix Gets All Three

- **Mutator performance** ← **bump allocation into blocks and holes**: objects allocated together sit together, like SS.
- **Fast collection** ← **mostly non-moving**: mark, then read the line map, like MS. Conservative marking keeps it cheap, defragmentation runs only on demand, in the same pass.
- **Space efficiency** ← **lines** reuse partly used blocks, **defragmentation** frees fragmented ones, and there is no 2× copy reserve.

# Conclusion

- **Immix:** mark-region with **blocks and lines**, plus **opportunistic defragmentation** that moves objects only when needed.
- **Faster than all canonical collectors** at every heap size, typically by **5–10%**.



# References


- Stephen M. Blackburn, Kathryn S. McKinley. *Immix: A Mark-Region Garbage Collector with Space Efficiency, Fast Collection, and Mutator Performance*. PLDI, 2008.